Exercise 2: Spring and Docker

Software Architecture

Winter Semester 2026/27

Tobias Arens M.sc.

Software Architecture Teaching and Research Unit
RWTH AACHEN UNIVERSITY

Overview

Spring

- Popular Java Framework
- Almost all heavy lifting runs with simple annotations
- Spring Boot: light-weight version of Spring with even less boilerplate code

Docker

- Containerized deployment
- Same environment everywhere
- Docker compose tool for easy orchestration of multiple services

- All examples in gitlab: https://gitlab.git.nrw/rwth-swa/teaching/swa-ws26-sample

Spring

Spring

What

- Popular Java framework often used for REST-backends
- „Inversion of Control“ (IoC)
- No manual instantiation of Classes, all managed by spring
- @Autowire annotation automatically connects components that depend on each other

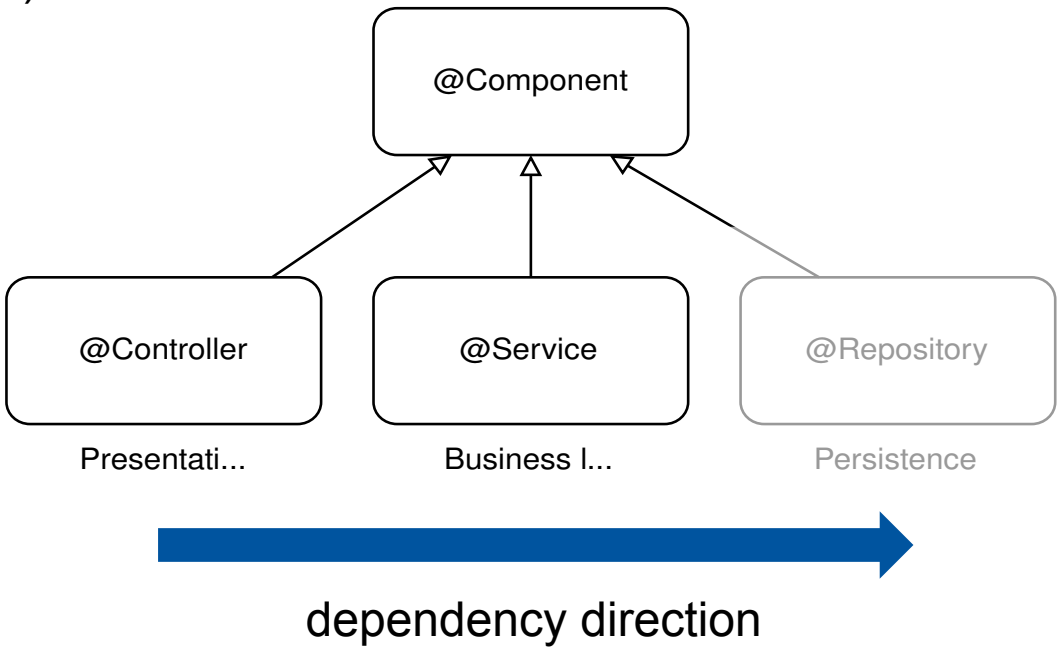
- Extremely feature rich and there are many extensions
  - Spring Boot: Includes embedded server, default configurations
  - Web MVC: Focused on Web apps
  - Many connectors to Databases



https://spring.io/

Idea

- Everything is organized in „Beans“
- Beans are components that can interact with each other and provide functionality
- Auto-detected on startup
- 3 main types (layers)



Service

- Most „plain“ component
- Contains Business logic for application

@Service-Bean
- Can be easily used by other components

```java
import org.springframework.stereotype.Service;

@Service  1 usage
public class HelloService {

    public String getGreeting() {  1 usage
        return "Hello";
    }

}
```

Spring Boot

(Rest)Controller

- Part of Spring MVC
- Presentation / Interface
- What endpoints do we provide

  - dependency automatically injected by spring
  - endpoint definition:
    - GetMapping: HTTP GET
    - PostMapping: POST
    - etc
  - Record as return value
    - will be converted to JSON

Implements an HTTP API

```java
@RestController
public class HelloController {

    private HelloService helloService;  2 usages

    @Autowired
    public HelloController(HelloService helloService) {
        this.helloService = helloService;
    }

    @GetMapping("/hello")
    public HelloAnswer hello() {
        var greeting = helloService.getGreeting();
        return new HelloAnswer(greeting);
    }

    public record HelloAnswer (String hello){};  2 usages

}
```

Spring Boot

Running

- Automatically starts webserver on default port 8080 (configurable)

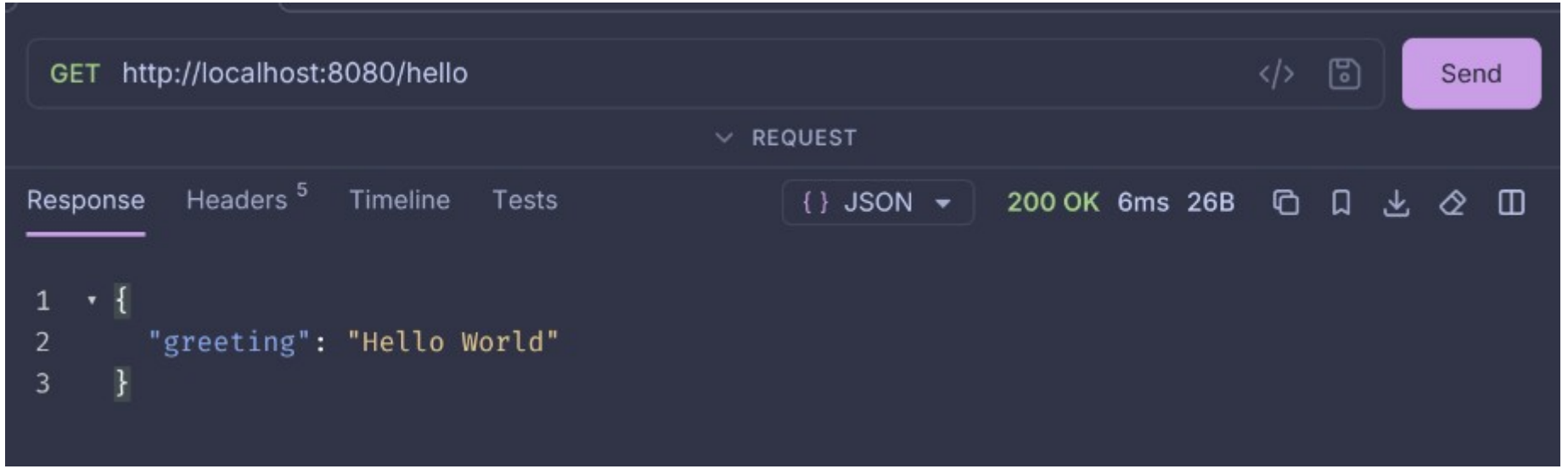
```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

2026-10-22T07:21:37.850Z  INFO 1 --- [swa-ws26-sample] [           main] s.r.de.sample.SampleApplication          :
 Starting SampleApplication v0.0.1-SNAPSHOT using Java 25.0.4.1 with PID 1 (/app/app.jar started by root in /app)
2026-10-22T07:21:37.855Z  INFO 1 --- [swa-ws26-sample] [           main] s.r.de.sample.SampleApplication          :
 No active profile set, falling back to 1 default profile: "default"
2026-10-22T07:21:38.563Z  INFO 1 --- [swa-ws26-sample] [           main] o.s.boot.tomcat.TomcatWebServer          :
 Tomcat initialized with port 8080 (http)
2026-10-22T07:21:38.577Z  INFO 1 --- [swa-ws26-sample] [           main] o.apache.catalina.core.StandardService   :
 Starting service [Tomcat]
2026-10-22T07:21:38.578Z  INFO 1 --- [swa-ws26-sample] [           main] o.apache.catalina.core.StandardEngine    :
```

Calling the HTTP API

- e.g. Firefox, Insomnia, Postman, Bruno



Example HTTP call in Bruno with JSON response

Build Process



https://maven.apache.org/

- We use apache maven
  - Build and dependency manager
  - Configuration pom.xml
- Reads the pom.xml, pulls all dependencies, and then builds a single runnable .jar file

```
<project>
<parent>...</parent>

<modules>...</modules>

<dependencies>
   <dependency>...</dependency>
   ...
</dependencies>

<build>...</build>

</project>
```

Example structure of a pom.xml

Docker

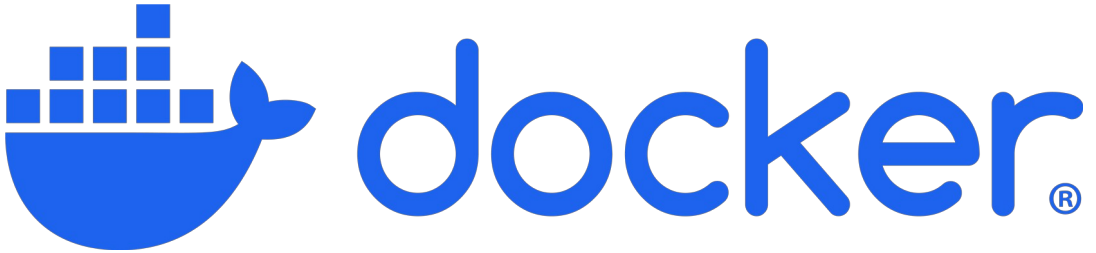
Docker

What

- Tool for running software in containers
  - Isolated environments that make it easy to install and run on different computers
  - No virtual machine!
- Virtual filesystem, network, process tree

Why

- No more „it works on my machine“
- easy setup
- same dependencies everywhere

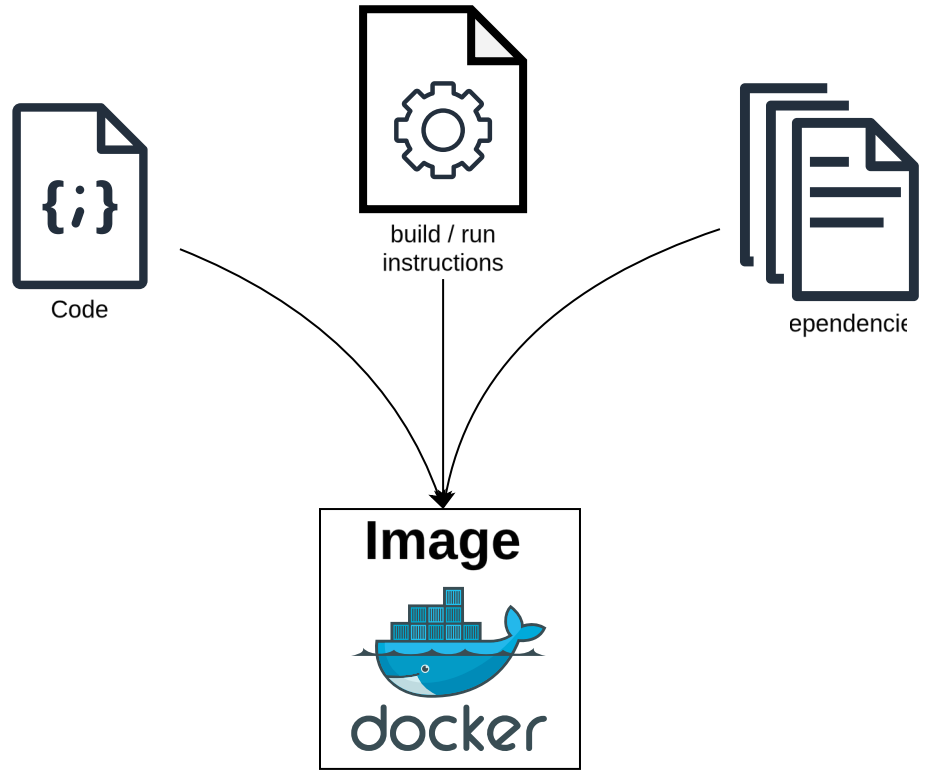


https://www.docker.com/

Docker

Basics

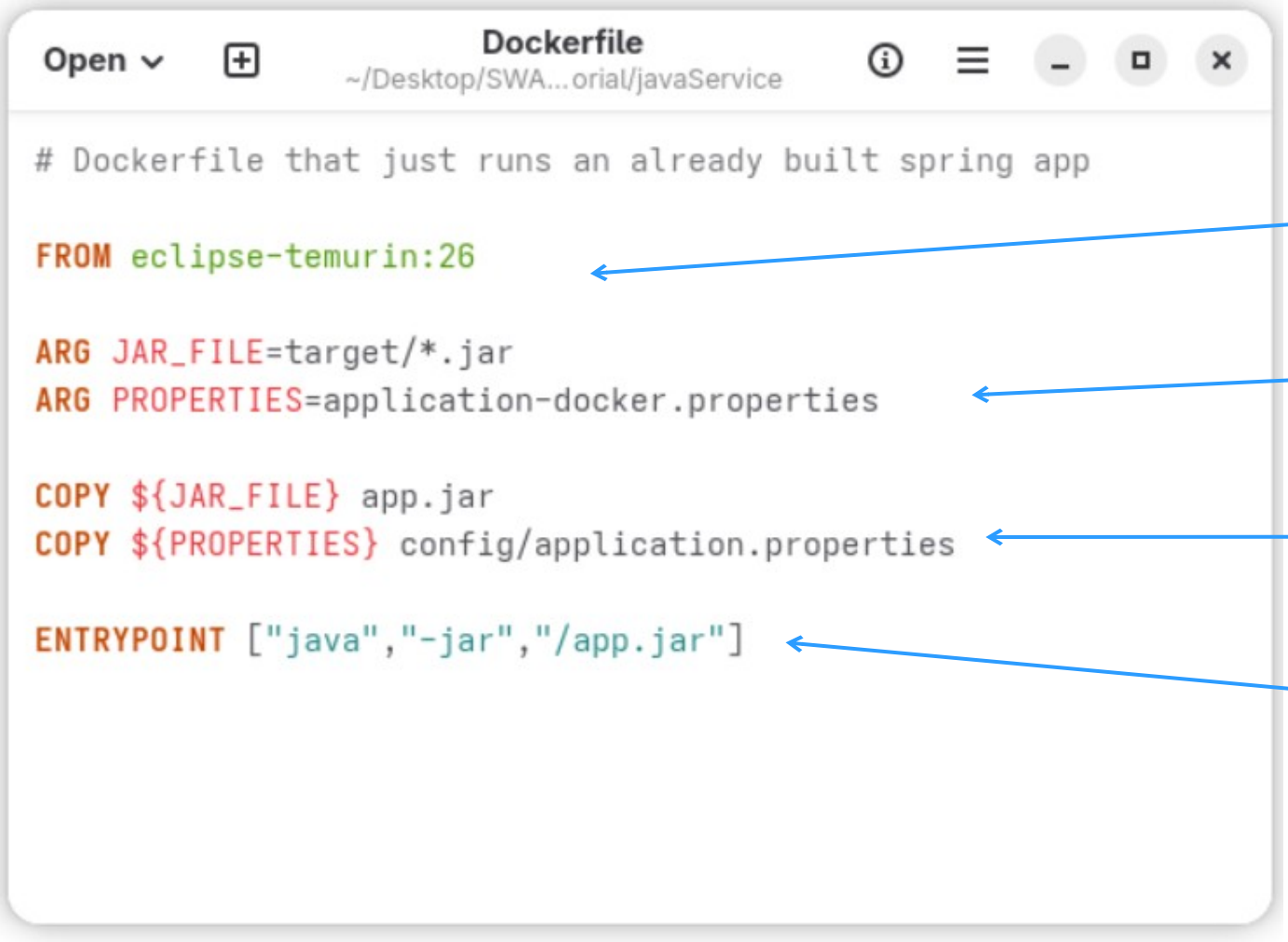
- *Image:* blueprint for running the software
- *Instance:* actual running container



Typical Workflow

- Write a Dockerfile in plain text
  - defines steps how to build your app
- Build image from dockerfile
- Run image
  - Specify which ports to use, which data volumes

Dockerfile – Example Java-Spring App (Execution)



base-image: preinstalled dependencies

specify variables

copy files into container

command for starting the app

Dockerfile – Example Python App (Execution)

Open ~/Desktop/SWA/teaching/... ckerTutorial/pythonService **Dockerfile**

```
FROM python:3.14

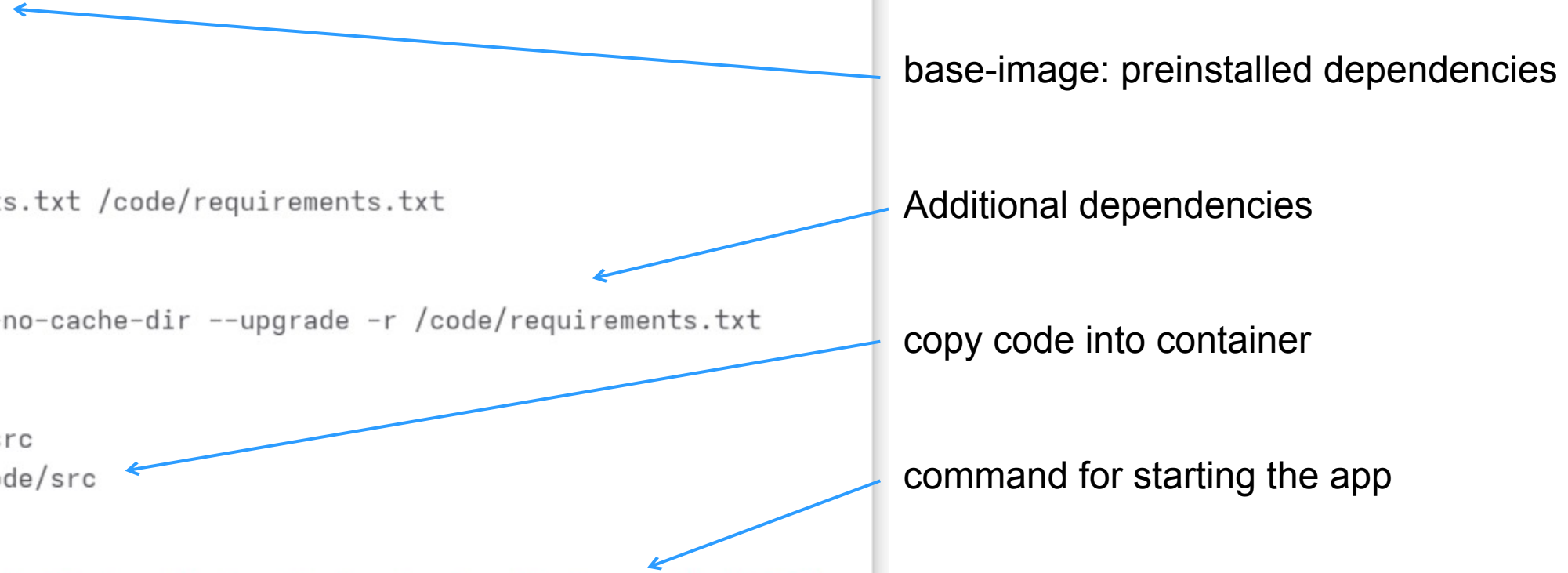
WORKDIR /code

COPY ./requirements.txt /code/requirements.txt

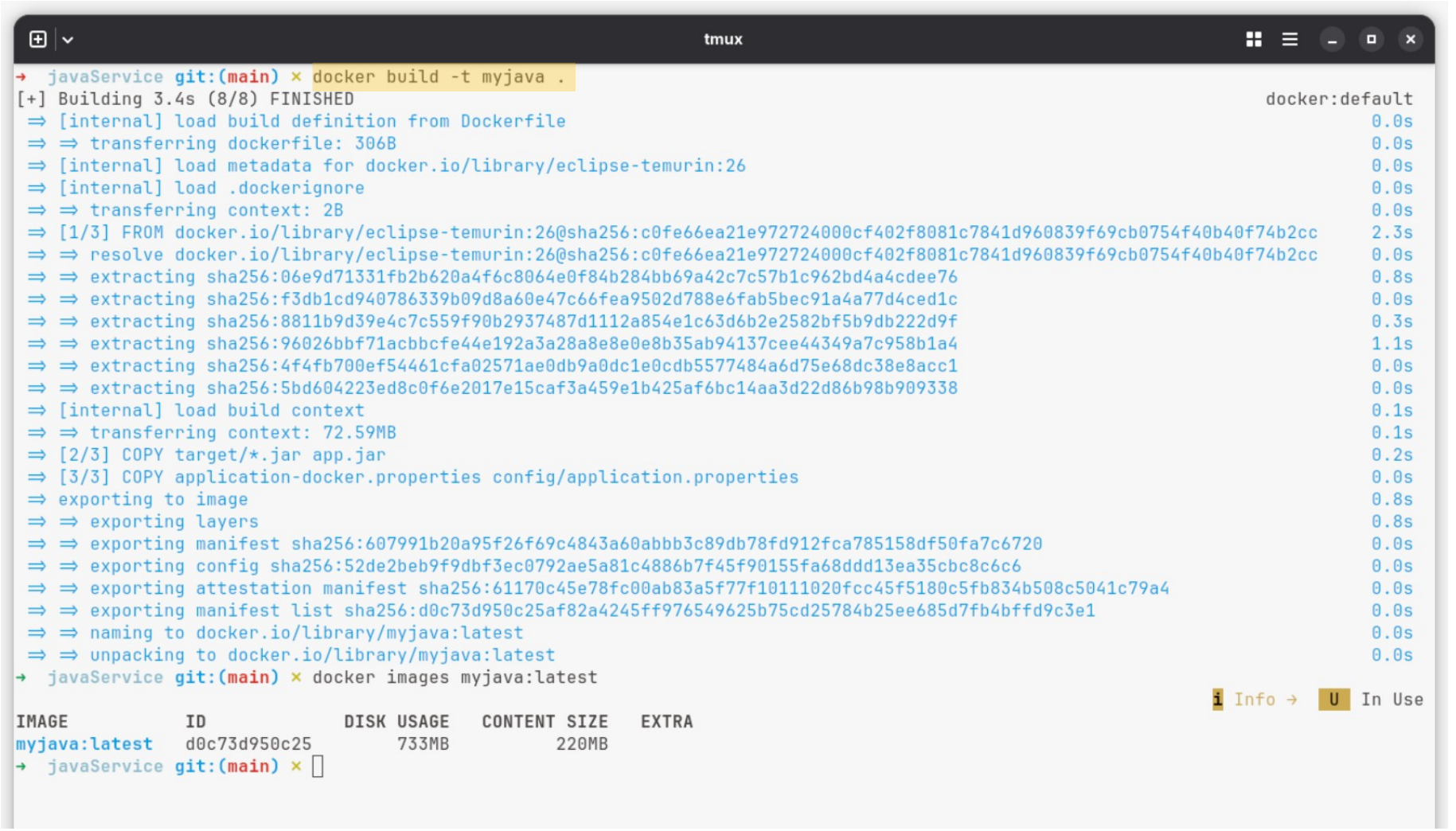
RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

COPY ./src /code/src
ENV PYTHONPATH=/code/src

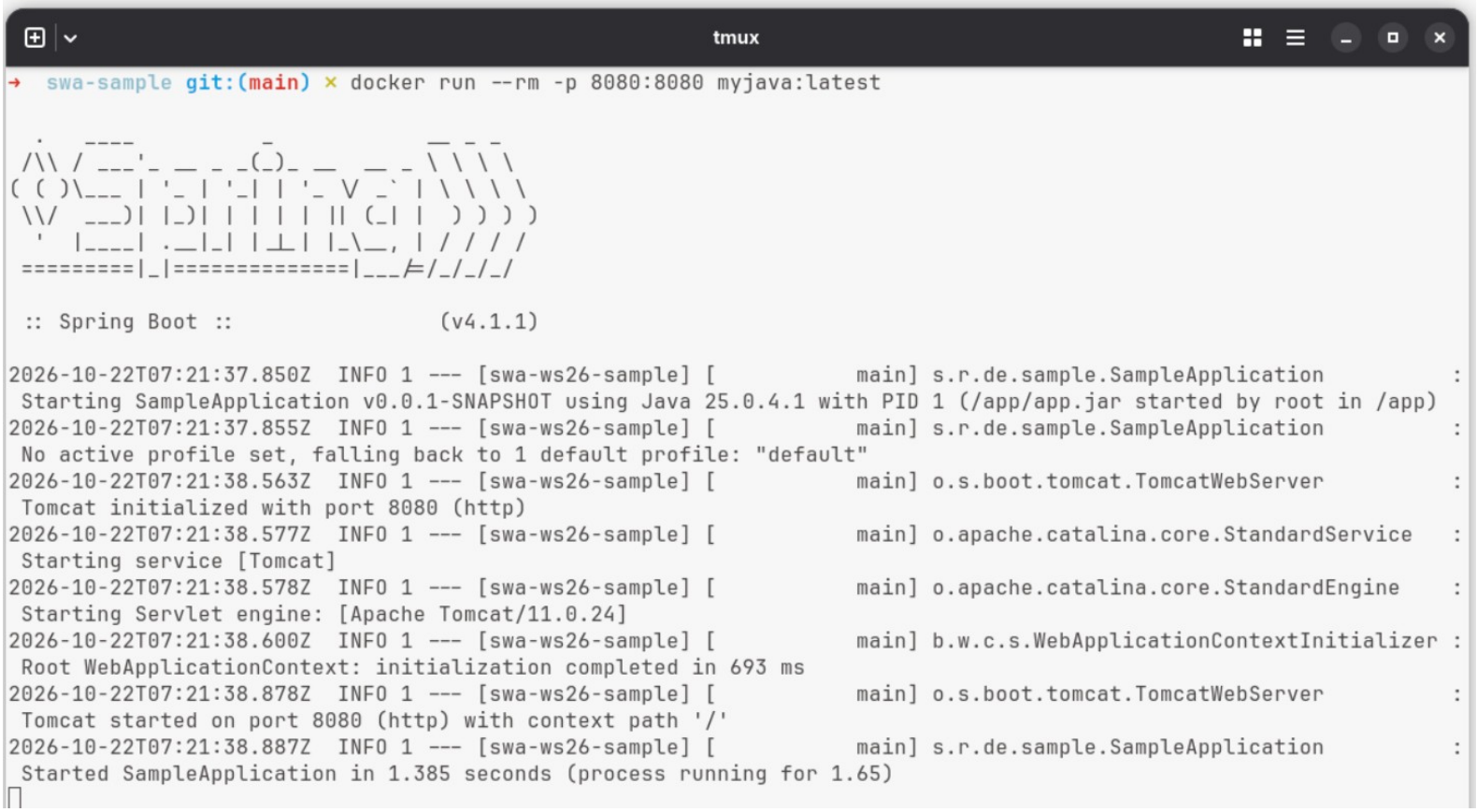
CMD ["python", "-m", "fastapi", "run", "src/main.py", "--port", "8000"]
```



Dockerfile – Example Usage



Dockerfile – Example Usage



Docker Compose

Docker Compose

Idea

- Compose a multi-component application from multiple different containers
- Configure containers / network / storage at once in a **single yaml text-file**

- One command `docker compose up` starts all services, `docker compose down` stops all
- Builds container if necessary, or forced rebuild with `--build`

Example



- Both are spring services running on 8080
- Both in the same maven project as modules
- In the sample project‘s git on branch `two_modules`

Docker Compose

Open • docker-compose.yml
~/Projects/swa-sample

```yaml
services:
  myservice-a:
    container_name: myservice-a
    build:
      context: ./service-a
      dockerfile: ../Dockerfile
    environment:
      SERVICE_B_URL: http://myservice-b:8080
    depends_on:
      - myservice-b
    ports:
      - "8081:8080"

  myservice-b:
    container_name: myservice-b
    build:
      context: ./service-b
      dockerfile: ../Dockerfile
    ports:
      - "8082:8080"
```

Two services

Build Instructions

Environment variables

Port mappings

Docker Port Mappings

